Unterstützung bei Arrays durch Java-API und IntelliJ

Einführung in die Programmierung 1

TU Wien, Wintersemester 25

Was wurde zuletzt besprochen?

- Arrays und Methoden
- Rekursion mit Arrays

Was wird in diesem Foliensatz besprochen?

- Hilfsklasse Arrays
- Unterstützung durch IDE
- Visualisierung von Arrays in der IDE

Was lernen Sie kennen?

- Hilfreiche Methoden aus der Java-API
- IDE-Unterstützung beim Debugging von Arrays
- Visualisierung von Arrays in der IDE

Hilfsklasse Arrays

Hilfsklasse für Arrays (muss importiert werden)

Nützliche überladene Methoden (hier für int)

- `static int binarySearch(int[] a, int key)`
  - Binäre Suche in einem Array
- `static boolean equals(int[] a, int[] a2)`
  - Vergleicht den Inhalt von zwei Arrays
- `static void fill(int[] a, int val)`
  - Befüllt alle Stellen eines Array mit einem bestimmten Wert
- `static void sort(int[] a)`
  - Sortiert den Inhalt eines Arrays aufsteigend
- `static String toString(int[] a)`
  - Liefert den Inhalt eines Arrays als formatierten String zurück

Beispiel (Methoden aus Arrays austesten)

```java
import java.util.Arrays;

public class ArraysTest {
    public static void main(String[] args) {
        int[] x = {3, 5, 2, 7, 9, 4, 6, 1, 8};
        boolean[] y = new boolean[3], z = new boolean[3];
        Arrays.sort(x);
        // java.util.Arrays.sort(x); impliziter Import
        System.out.println(Arrays.toString(x));
        System.out.println(Arrays.binarySearch(x, 9));
        Arrays.fill(y, true);
        Arrays.fill(z, true);
        System.out.println(Arrays.toString(y));
        System.out.println((y == z));
        System.out.println(Arrays.equals(y, z));
        y = z;
        System.out.println(y == z);
    }
}
```

```
[1, 2, 3, 4, 5, 6, 7, 8, 9]
8
[true, true, true]
false
true
true
```

Ausgabe mit deepToString

```
int[][] a = {{1, 2, 3}, {4, 5, 6}};
System.out.println(Arrays.deepToString(a));
```

```
[[1, 2, 3], [4, 5, 6]]
```

Für mehrdimensionale Arrays geeignet!

Vergleich mit deepEquals

```
int[][] a = {{1, 2, 3}, {4, 5}};
int[][] b = {{1, 2, 3}, {4, 5}};
int[][] c = {{1, 2, 3}, {4, 5, 6}};
System.out.println(Arrays.deepEquals(a, b));
System.out.println(Arrays.deepEquals(a, c));
```

```
true
false
```

Für mehrdimensionale Arrays geeignet!

Unterstützung im Debugger (1)

```
9   for (int i = 0; i < n; i++) {
10      System.out.print(data[i] + "\t");   data: [84, 65, 56, 39, 40, 90, 8, 35, 89, 61]
11  }
12  System.out.println();
13  for (int i = 0; i < n / 2 ; i++) {   n: 10
14      int temp = data[i];
15      data[i] = data[n - i - 1];
16      data[n - i - 1] = temp;
17  }
18  for (int i = 0; i < n; i++) {
```

Inhalt des Arrays inspizieren

```
"main"@1 in group "main": RUNNING
main:13, ArrayTest2
Evaluate expression (Eingabe) or add a watch (Strg+Umschalt+Eingabe)
args = {String[0]@804} []
n = 10
data = {int[10]@805} [84, 65, 56, 39, 40, 90, 8, 35, 89, 61]
  0 = 84
  1 = 65
  2 = 56
  3 = 39
  4 = 40
  5 = 90
  6 = 8
  7 = 35
  8 = 89
  9 = 61
```

Inhalt des Arrays inspizieren (hier mit Index)

Unterstützung im Debugger (2)

```
public class Array2D {

    public static void main(String[] args) {   args: []
        int[][] a = new int[2][3];   a: int[2][]@789
        for (int i = 0; i < a.lengt
            for (int j = 0; j < a[i
                a[i][j] = (int) (Ma
            }
        }
        for (int i = 0; i < a.lengt
            for (int j = 0; j < a[i
```

```
a = {int[2][]@789}
  0 = {int[3]@792} [0, 3, 9]
    0 = 0
    1 = 3
    2 = 9
  1 = {int[3]@793} [9, 2, 6]
    0 = 9
    1 = 2
    2 = 6
Set value F2   Create renderer   Add as inline watch
```

2D-Arrays aufklappen und inspizieren

```
"main"@1 in group "main": RUNNING
main:10, Array2D
Evaluate expression (Eingabe) or add a watch (Strg+Umschalt+Eingabe)
args = {String[0]@788} []
a = {int[2][]@789}
  0 = {int[3]@792} [0, 3, 9]
    0 = 0
    1 = 3
    2 = 9
  1 = {int[3]@793} [9, 2, 6]
    0 = 9
    1 = 2
    2 = 6
a.length = 2
```

2D-Arrays aufklappen und inspizieren

Java Visualizer Plugin

IntelliJ kann mit Plugins erweitert werden

Java Visualizer Plugin erlaubt Visualisierung von Arrays

Java Visualizer Plugin

```
public class Array2D {

    public static void main(String[] args) {   args: []
        int[][] a = new int[2][3];   a: int[2][]@789
        for (int i = 0; i < a.length; i++) {
            for (int j = 0; j < a[i].length; j++) {
                a[i][j] = (int) (Math.random() * 10);
            }
        }
        for (int i = 0; i < a.length; i++) {   a: int[2][]@789
            for (int j = 0; j < a[i].length; j++) {
                System.out.print(a[i][j] + " ");
            }
            System.out.println();
```

Stack

Call Stack

main:10

a

Objects

int[][]

0 1

int[]

0 1 2

0 3 9

int[]

0 1 2

9 2 6

Einzelne Arrays mit Referenzen